

PROGRAM DSA/小班课

[DSA 课程网站](#)

常欣海

PROGAM 教学计划

Course Syllabus

数算 B 小班课教学计划

周次	日期	时间	主题
3	3 月 21 日周六	18:40-20:30	基础、复杂度与线性表
6	4 月 11 日周六	18:40-20:30	链表、串与线性结构应用
11	5 月 16 日周六	18:40-20:30	排序、树与检索
12	5 月 23 日周六	18:40-20:30	图算法与综合收束
13	5 月 30 日周六	18:40-20:30	期末上机复习
14	6 月 6 日周六	18:40-20:30	期末笔试复习

PROGRAM

L4 图 与图算法

常欣海

2026.05.23

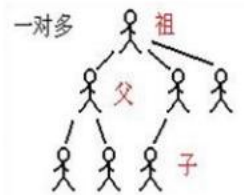
什么是“图”

Graph

数据的逻辑结构

- 线性结构：唯一前驱，唯一后继，线性关系
- 树结构：唯一前驱，多个后继，层次关系
- 图结构：多个前驱、多个后继，网状关系

- 图的逻辑结构： $G = (V, E)$
 - 图由顶点集合与边集合组成
 - V 为有穷的顶点集合
 - E 为边集合，是顶点的偶对（边的始点，边的终点）集合
 - 用 $|V|$ 表示顶点的总数， $|E|$ 表示边的总数

	集合	线性结构	树形结构	图状或网状结构
特征	元素间为松散的关系	元素间为一对一关系	元素间为一对多关系	元素间为多对多关系
示例	同属色彩集合 	$a \rightarrow b \rightarrow c \rightarrow d$		 公路交通网

什么是“图”

Graph

术语

解释

图 (Graph)

由顶点集 V 和边集 E 组成的结构，记为 $G = (V, E)$ 。

顶点 / 节点 (Vertex / Node)

图的基本单元。

边 (Edge)

连接两个顶点的线。

有向图 (Directed Graph)

边有方向（箭头），从 u 指向 v 记为 (u, v) 。

无向图 (Undirected Graph)

边无方向， $\{u, v\}$ 或 (u, v) 视为双向。

加权图 (Weighted Graph)

每条边带有一个数值（权值）。

简单图 (Simple Graph)

无自环、无重边的图。

多重图 (Multigraph)

允许两顶点间有多条边（重边），允许自环。

什么是“图”

Graph

术语

解释

度 (Degree)

顶点关联的边数。有向图中分为入度 (indegree) 和出度 (outdegree)。

自环 (Loop)

起点和终点相同的边。

邻接 (Adjacent)

两顶点之间有边相连。

孤立点 (Isolated Vertex)

度为 0 的顶点。

悬挂点 (Pendant Vertex)

度为 1 的顶点。

什么是“图”

Graph

路径 (Path)

顶点序列 $v_0, v_1, \dots, v_k$ ，相邻顶点间有边，且顶点互不相同（简单路径）。

回路 / 圈 (Cycle)

起点与终点相同的路径，且其他顶点不重复。

长度 (Length)

路径上的边数（或权和）。

连通图 (Connected Graph)

任意两顶点之间都有路径。

连通分量 (Connected Component)

极大连通子图。

强连通 (Strongly Connected)

有向图中，任意两顶点 u, v 既存在 $u \rightarrow v$ 路径也存在 $v \rightarrow u$ 路径。

弱连通 (Weakly Connected)

将有向边视为无向边后连通。

桥 (Bridge)

删除后使图不连通的边。

割点 (Articulation Point)

删除后使图不连通的顶点。

什么是“图”

Graph

例子

1 社交网络

用户为结点，好友/关注关系为边



2 交通网络

城市、站点或路口为结点，道路/航线/线路为边



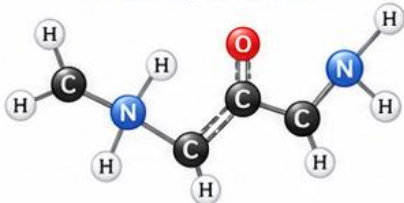
3 知识图谱

实体为结点，语义关系为边



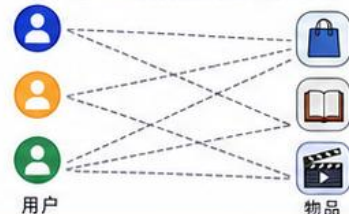
4 分子图

原子为结点，化学键为边



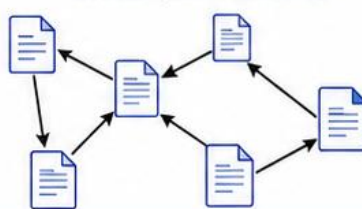
5 推荐系统

用户—物品关系常表示为二部图



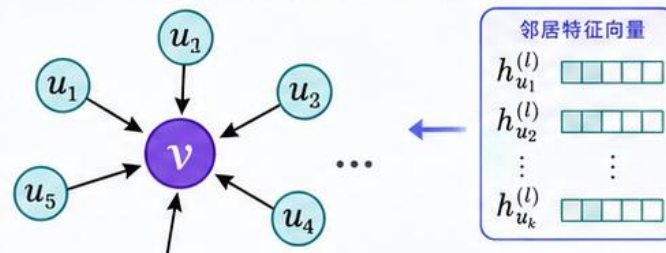
6 引文网络

论文为结点，引用关系为有向边



GNNs (图神经网络)

GNN 将图结构与结点特征结合，通过邻居信息聚合学习结点表示。



$$h_v^{(l+1)} = \text{AGG}(\{h_u^{(l)} : u \in \mathcal{N}(v)\})$$

典型任务



节点分类



链路预测



图分类

很多现实系统都可以抽象为图：对象是节点 (node)，关系是边 (edge)



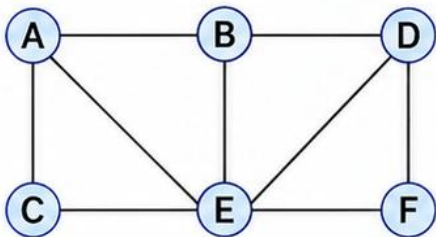
定义

“图”的定义

Graph

图表示: $G = (V, E)$

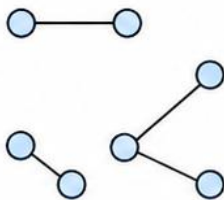
- V : 顶点 (Vertex) 集合, 也记作 $|V| = n$
- E : 边 (Edge) 集合, 也记作 $|E| = m$
- 边 $e = (u, v)$ 表示顶点 u 与 v 之间的连接关系



$V = \{A, B, C, D, E, F\}$
 $E = \{(A, B), (A, C), (B, D), (B, E), (C, E), (D, F), (E, F)\}$

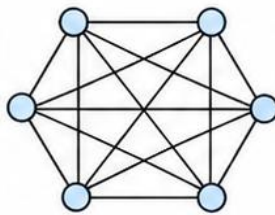
1 稀疏图

边较少, $|E|$ 远小于 $|V|^2$



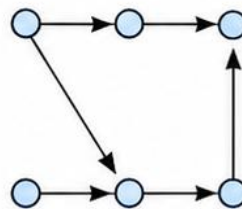
2 稠密图

边较多, 接近完全连接



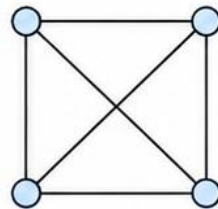
3 有向图

边有方向, 用箭头表示



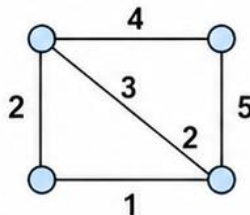
4 无向图

边无方向



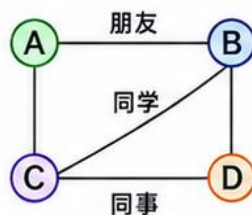
5 带权图

边带数值权重, 如距离、代价



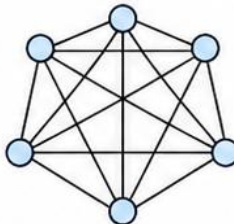
6 标号图

结点或边带标签, 如类别、名称



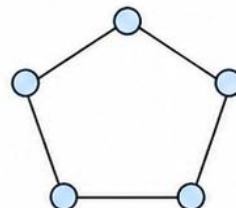
7 完全图

任意两个顶点之间都有边



8 简单图

无自环、无重边

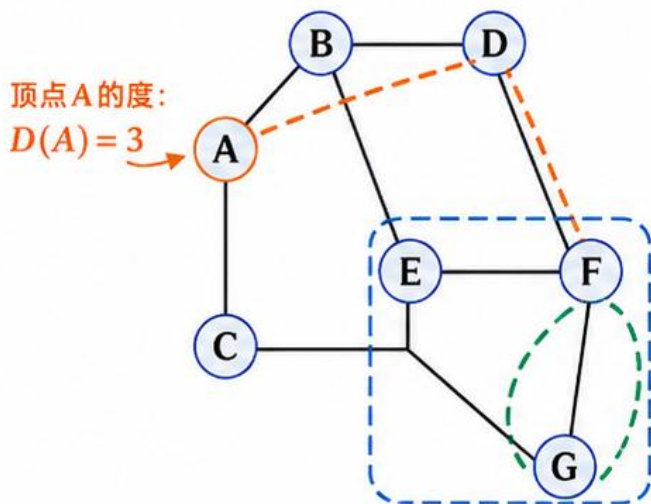


“图”的定义

Graph

性质

综合示意图（无向图示例）



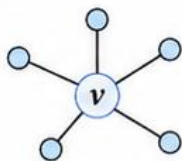
--- 路径示例: $A \rightarrow B \rightarrow D \rightarrow F \rightarrow G$

--- 回路/环: $D \rightarrow F \rightarrow G \rightarrow D$

子图: $\{E, F, G\}$ 及其相关边

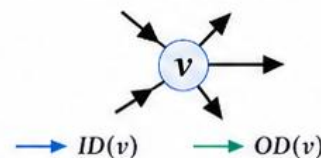
顶点度数（无向图）

$D(v)$ = 与 v 相连的边数



入度 / 出度（有向图）

入度 $ID(v)$ 为指向 v 的边数
出度 $OD(v)$ 为从 v 指出的边数



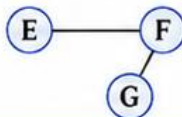
相邻结点

若两点之间有边，
则互为相邻



子图

由原图的部分顶点和
边构成



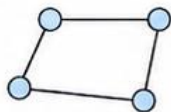
路径 (Path)

顶点序列 $v_1 \rightarrow v_2 \rightarrow \dots \rightarrow v_k$ ，
相邻顶点连续相连



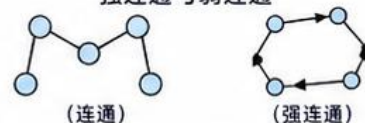
回路 / 环 (Cycle)

起点和终点相同的路径，
且中间顶点不重复



连通性

无向图中任意两点间存在
路径则连通；有向图可区分
强连通与弱连通



常见术语总结

术语	含义	直观理解
顶点 (点)	图中的基本对象	如人、城市、网页
边 (边/弧)	连接两点的关系	如道路、通信、引用
度 $D(v)$	与 v 相连的边数	反映重要性/活跃度
入度 ID	指向 v 的边数	有多少进入 v
出度 OD	从 v 指出的边数	有多少从 v 出去
相邻	两点之间有边	彼此直接连接
路径	顶点序列	经过的一系列点
回路/环	起终点相同的路径	形成闭合圈
子图	原图的部分点和边	原图的局部
连通	任意两点有路径	可以互相到达

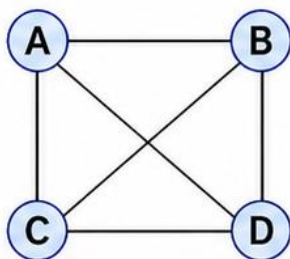
“图”的定义

Graph

存储

邻接矩阵 (Adjacency Matrix)

示例图 (4 个顶点)



邻接矩阵 ($A[i][j]$)

	A	B	C	D
A	0	1	1	0
B	1	0	0	1
C	1	0	0	1
D	0	1	1	0

若顶点 i 与 j 相连, 则矩阵 $A[i][j] = 1$;
否则为 0 (无权图示例)

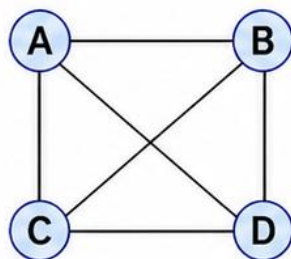
✓ 优点: 查询边是否存在很快, 适合稠密图

✗ 缺点: 空间复杂度高, 稀疏图不经济

空间复杂度 $O(|V|^2)$

邻接表 (Adjacency List)

示例图 (4 个顶点)



邻接表表示

	邻接表表示
A	B, C
B	A, D
C	A, D
D	B, C

对每个顶点记录其所有邻居,
更适合边较少的图

✓ 优点: 节省空间, 适合稀疏图

✗ 缺点: 判断某条边存在性通常较慢

空间复杂度 $O(|V| + |E|)$



稀疏图更常用
邻接表

两种存储方式对比

方式	核心思想	适用场景	查询边	空间开销
邻接矩阵	用二维矩阵记录边	稠密图	快	大
邻接表	用链表或列表记录邻居	稀疏图	较慢	小



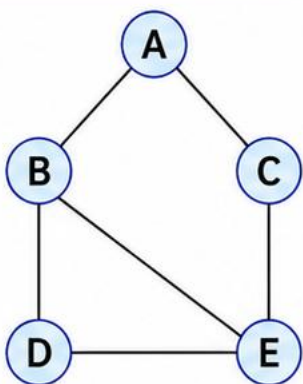
兼顾查询效率
与空间成本

“图”的定义

Graph

遍历

示例图（无向图）

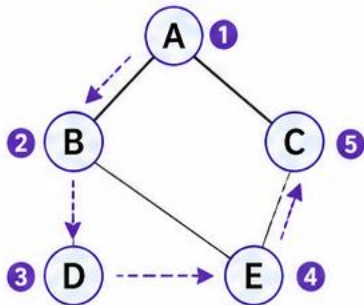


$V = \{A, B, C, D, E\}$
 $E = \{(A, B), (A, C), (B, D), (B, E), (C, E), (D, E)\}$

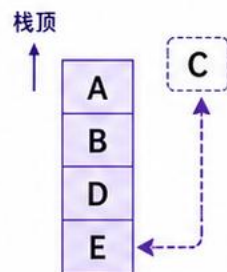
DFS（深度优先搜索）

一条路走到尽头，再回溯

遍历过程示意



访问过程（递归 / 栈）



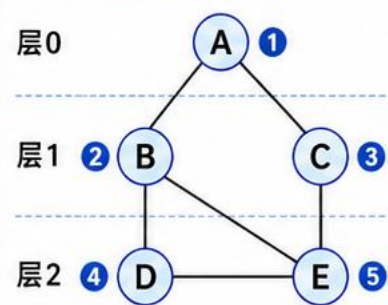
访问顺序 $A \rightarrow B \rightarrow D \rightarrow E \rightarrow C$

适用场景 适合连通分量、拓扑相关搜索、回溯问题

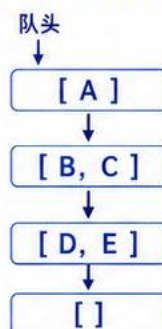
BFS（广度优先搜索）

按层逐步向外扩展

分层扩展示意



分层扩展（队列）



访问顺序 $A \rightarrow B \rightarrow C \rightarrow D \rightarrow E$

适用场景 适合最短路径（无权图）、层次遍历

DFS 与 BFS 对比

策略	数据结构	访问特点	典型用途
DFS（深度优先）	栈（Stack） / 递归	尽可能深入，遇阻回溯	连通分量、拓扑排序、回溯搜索
BFS（广度优先）	队列（Queue）	按层扩展，先近后远	最短路径（无权图）、层次遍历

“图”的定义

遍历

Graph

深度优先搜索

在图G中寻找一条由start到end的路径 path, 不存在则返回 None

```
def dfs(G, start, end, visited, path):
```

将所有顶点设置为未访问

将 start 结点添加到 path 列表中, 并设置为已访问

```
if start == end:
```

```
    return path
```

对于 start 结点的每个邻居 neighbor:

如果未曾访问 neighbor:

```
    result = dfs(G, neighbor, end, visited, path)
```

如果 result 非空, 意味着搜索成功, return result

搜索失败, return None

递归版本

```
def dfs_recursive(graph, node, visited=None):
```

```
    if visited is None:
```

```
        visited = set()
```

```
    visited.add(node)
```

```
    print(node, end=' ')
```

```
    for neighbor in graph[node]:
```

```
        if neighbor not in visited:
```

```
            dfs_recursive(graph, neighbor, visited)
```

```
    return visited
```

迭代版本 (用栈模拟递归)

```
def dfs_iterative(graph, start):
```

```
    visited = set()
```

```
    stack = [start]
```

```
    order = []
```

```
    while stack:
```

```
        node = stack.pop() # 栈顶弹出
```

```
        if node in visited:
```

```
            continue
```

```
        visited.add(node)
```

```
        order.append(node)
```

```
        for neighbor in reversed(graph[node]): # reversed 保持与递归版一致
```

```
            if neighbor not in visited:
```

```
                stack.append(neighbor)
```

```
    return order
```

“图”的定义

Graph

遍历

广度优先搜索

在图 G 中寻找一条由 start 到 end 的路径，不存在则返回 None

def bfs(G, start, end):

 将所有顶点初始化为未访问，父结点初始化为空，初始化队列为空

 将 start 标记为已访问，并将 start 入队

 while 队列非空:

 将队头元素赋值给 current_node

 if current_node == end: 退出循环

 对 current_node 的每个邻居 v:

 若 v 未被访问，将 v 入队，并将 current_node 标记为已访问

 记录 v 的父节点为 current_node

 if end 结点未被访问: 搜索失败 return None

 依据记录的每个结点的父结点，从终点开始逆向重构路径并返回

===== 广度优先搜索 BFS =====

from collections import deque

def bfs(graph, start):

 visited = set([start])

 queue = deque([start])

 order = []

 while queue:

 node = queue.popleft() # 队首弹出

 order.append(node)

 for neighbor in graph[node]:

 if neighbor not in visited:

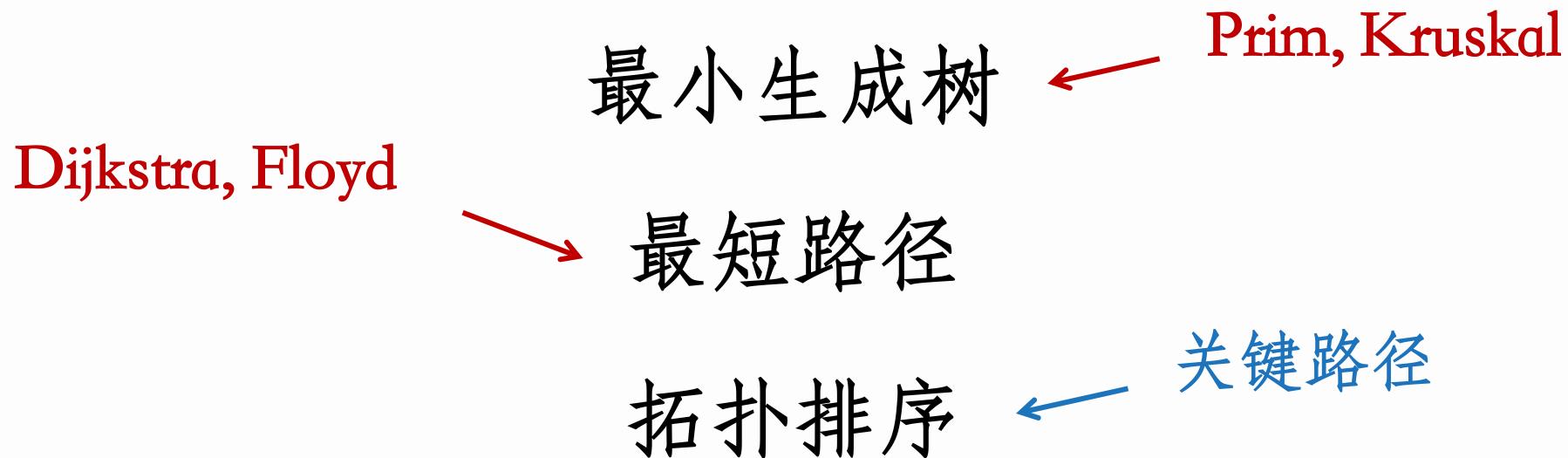
 visited.add(neighbor) # 入队时即标记，防止重复入队

 queue.append(neighbor)

 return order

“图”的应用

Graph



什么问题？ 解决方法？ 基本原理/正确性证明？ 代码/复杂度？

“图”的应用

Graph

最小生成树

- 定义：生成树（Spanning Tree）
 - 对于带权无向连通图 $G=(V, E)$ ，以及 G 的一个子图 $G'=(V', E')$ ，若 $V=V'$ ，且 G' 是不含回路的连通图，则称 G' 是 G 的生成树
 - 连通图的生成树是连通图的一个极小连通子图，它含有图中的全部 n 个顶点，以及足以构成一棵树的 $n-1$ 条边。
 - 增加一条边，则必定构成环；
 - 去掉一条边，则连通图变为不连通的。
 - 基于 DFS 以及 BFS 得到的搜索树，就是图的生成树，因此也称为 DFS 生成树、BFS 生成树
- 定义：最小生成树（Minimum Spanning Tree, MST）
 - 定义生成树的权值为其中所有边的权值之和，则 G 的所有生成树中，权值最小的生成树称为图 G 的最小生成树

“图”的应用

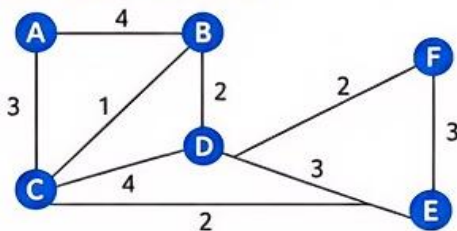
最小生成树

Graph

1 问题引入

🏗️ 铺设道路 / 电网 / 通信网络时，希望连接所有城市，同时总成本最低。

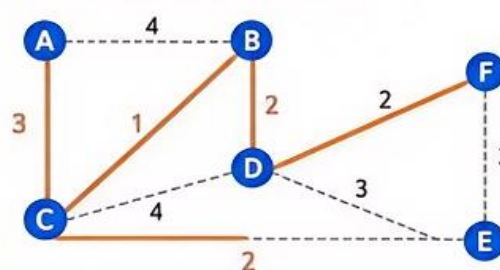
■ 原图（带权无向连通图）



- 边权表示成本、距离或代价
- 目标：选出若干条边，使所有顶点连通
- 同时不能形成回路，并且总权值最小

🎯 **核心问题：**在“连通”与“最省”之间找到最佳平衡。

■ 一棵最小生成树 (MST) 示例



选中的边：

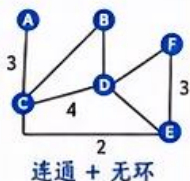
BC = 1
BD = 2
CE = 2
DF = 2
AC = 3

总权值 = 10

2 定义

1 生成树

包含原图全部顶点的连通无环子图。



连通 + 无环

2 最小生成树 (MST)

在所有生成树中，边权和最小的那一棵（或若干棵）。

$$w(T) = \sum_{e \in T} w(e) \text{ 最小}$$



权值和最小

3 判定条件

- 覆盖所有顶点
- 边数恰好为 $n-1$
- 不存在环
- 总权值最小

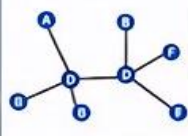


❗ **MST 只适用于带权、无向、连通图。**

3 重要性质

性质 1 边数性质

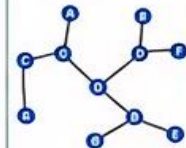
任意一棵生成树都有 $|V|-1$ 条边。



当 $|V| = n$ 时，边数 $= n-1$

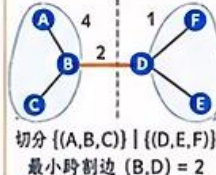
性质 2 唯一性

若图中所有边权互不相同，则 MST 唯一。



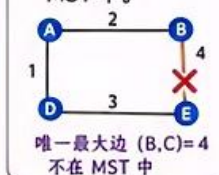
性质 3 切分性质 (Cut Property)

对任意切分，跨越该切分的最小权边一定属于某棵 MST。



性质 4 环性质 (Cycle Property)

在任意环中，若某条边是唯一最大边，则它不可能出现在 MST 中。



★ **MST 可能不唯一，但最小总权值相同。**

“图”的应用

最小生成树

Graph

4 算法一览

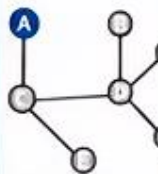


Prim 算法：从点出发，逐步扩张

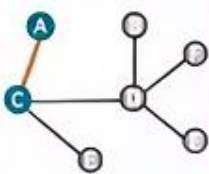


核心思想：从任意起点开始，维护已选顶点集合 S ；每次选择一条连接 S 与 $V-S$ 的最小权边，将新顶点加入。

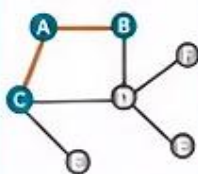
Step 1
选起点 A



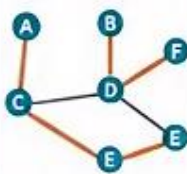
Step 2
选最小跨界边
 $AC = 3$



Step 3
再选 $BC = 1$
或 $CE = 2$ 等



Step 4
逐步扩张直到
所有点加入



复杂度

邻接矩阵: $O(n^2)$; 堆优化: $O(m \log n)$



适用场景

适合稠密图



本质：不断扩张一棵树。



Kruskal 算法：从边出发，逐步合并

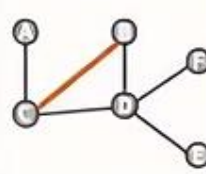


核心思想：先将所有边按权值从小到大排序；依次尝试加入当前最小边，若不会形成环则保留，否则跳过。

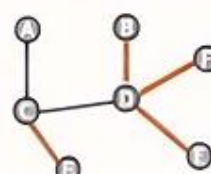
Step 1
边排序（按权值升序）

BC	1
BD	2
CE	2
DF	2
AC	3
DE	3
EF	3
AB	4
CD	4

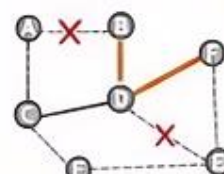
Step 2
选最小边
 $BC = 1$



Step 3
选 $BD = 2$ 、
 $CE = 2$ 、 $DF = 2$



Step 4
选 $AC = 3$ 得到 MST；
成环边跳过（打叉）



复杂度

排序主导: $O(m \log m) \approx O(m \log n)$



数据结构

常与并查集 (Union-Find) 配合



适用场景

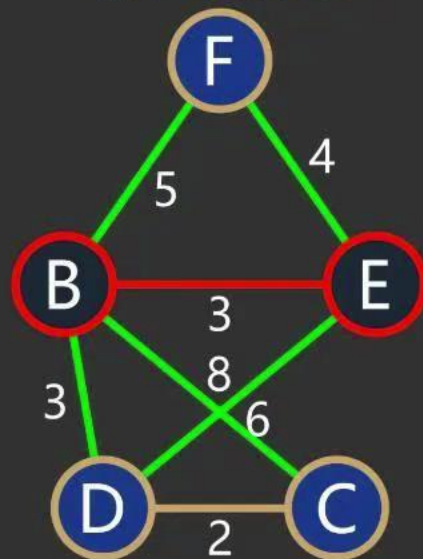
适合稀疏图



本质：不断连接多个连通分量。

Prim算法

动画演示



图论系列

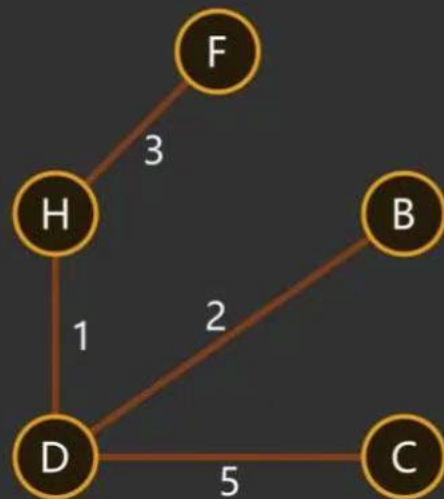
30 秒彻底搞懂最小生成树



面向初学者|简单又好懂

Kruskal算法

最小生成树原来这么简单？



图论系列

贪心算法+并查集这样用！



面向初学者|简单又好懂

“图”的应用

最小生成树

Graph

通用算法：初始时，每个顶点是蓝色集合（或红色集合），所有边无色。

重复以下两条规则之一（直到所有顶点属于同一集合）：

1. **蓝色规则：**选择连接两个不同蓝色集合的**最小权边**，将其染成蓝色（加入 MST）。
2. **红色规则：**选择同一蓝色集合内部的**最大权边**（在某个环上），将其染成红色（丢弃）。

可以证明：无论以何种顺序应用这些规则，最终蓝色边构成一棵 MST。

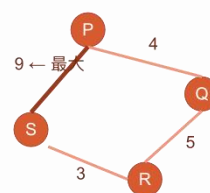
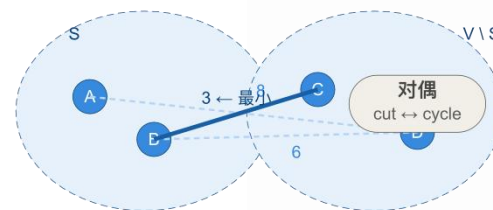
- **Prim 算法：**始终应用蓝色规则，但只有一个蓝色集合是“活”的（当前树），其余顶点各自为蓝色单点集合。每次从当前树与外部的最小边入手，将新顶点并入。
- **Kruskal 算法：**也始终应用蓝色规则，但所有蓝色集合平等，初始每个顶点是一个蓝色集合。每次全局找最小权边，若两端不在同一蓝色集合中，则染蓝并合并两个集合。
- **Borůvka 算法**（见后文）：同时为**每个蓝色集合**找出连接该集合与外部的**最小边**，然后同时将这些边染蓝并合并集合。并行性更强。

蓝色规则（切割定理）

割中最小边 → 必入 MST

红色规则（环路定理）

环上最大边 → 不入任何 MST



两条规则共同构成 MST 边集的充要刻画
蓝色边集 = MST，红色边集 = 非 MST 边

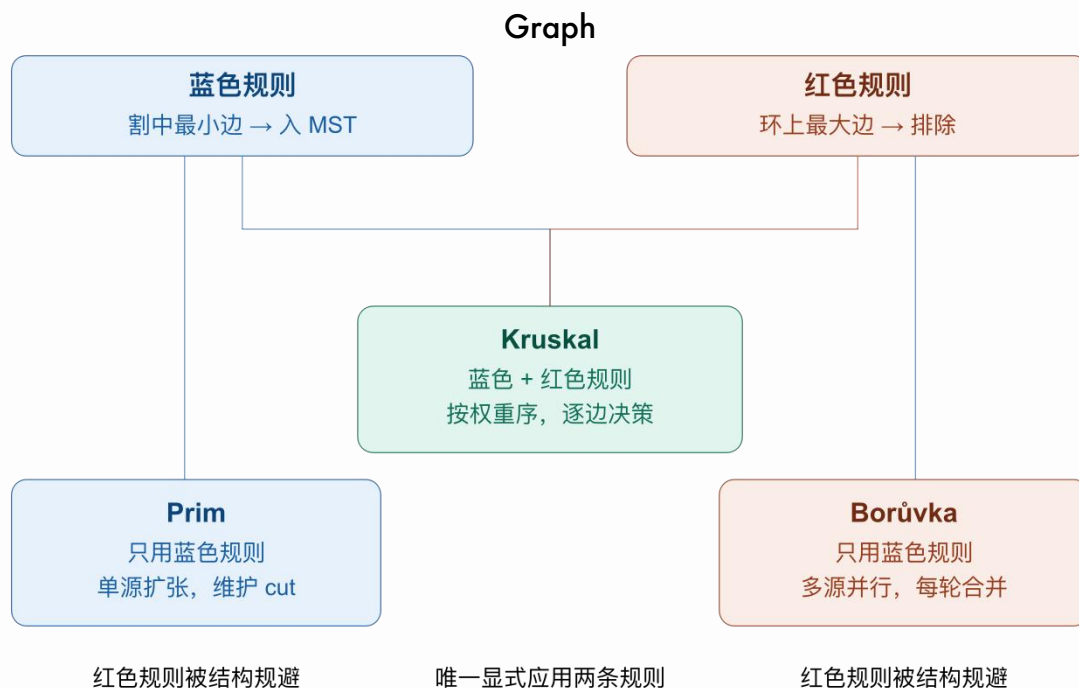
蓝红规则

“图”的应用

最小生成树

蓝红规则

可视化



算法	维护的对象	关键操作	适用图
Prim	一棵树 + 未加入顶点	取与树相邻的最小边（优先队列 / 邻接矩阵）	稠密图
Kruskal	所有连通分量（并查集） + 全局边排序	排序后依次取边，判断是否同分量	稀疏图
Borůvka	所有连通分量（并查集）	每轮为每个分量找最小外出边（需要邻接表）	并行 / 分布式 / 特殊结构

最短路径

“图”的应用

Graph

最短路径问题研究如何在带权图中找到总代价最小的路径，是网络路由、地图导航、任务规划与图计算中的基础问题。

1 问题引入



- 在通信网络中，数据从源主机到目标主机需要经过多跳转发。
- 每条边都可能具有代价，例如距离、时间、带宽消耗或金钱成本。
- 因此可以将系统抽象为带权图，并寻找“总代价最小”的路径。

2 形式化定义

- 给定带权图 $G = (V, E, w)$ ，其中 $w(e)$ 表示边 e 的权值。
- 对于一条路径 $P = (v_0, v_1, \dots, v_k)$ ，其长度定义为各边权之和：

$$\text{len}(P) = \sum_{i=0}^{k-1} w(v_i, v_{i+1})$$

- 从顶点 s 到顶点 t 的最短路径，是所有 $s \rightarrow t$ 路径中长度最小的一条。
- 记距离为 $\text{dist}(s, t)$ 。

小问题分类



单源最短路径 (SSSP)
从源点 s 到所有顶点



单点对最短路径
从 u 到 v



单目的地最短路径
所有点到 t



所有点对最短路径 (APSP)
任意 u, v 之间

3 关键性质



最优子结构

若 P 是 $s \rightarrow t$ 的最短路径，则 P 的任意子路径也是对应端点之间的最短路径。



简单路径性质

最短路径可视为简单路径；
正权回路不会出现在最短路中，
零权回路可删除而不改变长度。



负权边与负回路

Dijkstra 要求边权非负；若存在可达负回路，则某些最短路径可能不存在。Floyd/Bellman-Ford 可用于处理或检测相关情况。

“图”的应用

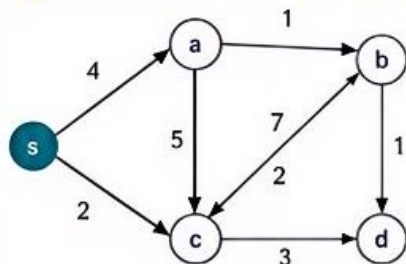
最短路径

Graph

4 算法总览

算法	适用场景	边权条件	时间复杂度	特点
BFS	无权图/等权图	边权相等	$O(V + E)$	按层扩展, 求最少边数/等权最短路
Dijkstra	单源最短路径	非负边权	$O((V + E) \log V)$	贪心 + 优先队列, 高效实用
Bellman-Ford	单源最短路径	允许负权边	$O(V E)$	可检测负回路, 但更慢
Floyd-Warshall	所有点对最短路径	允许负权边 但不能有负回路	$O(V ^3)$	动态规划, 适合稠密图与全局距离计算

5 Dijkstra: 单源最短路径的经典贪心算法



- 1 初始化: $\text{Dist}[s] = 0$, 其余为 ∞ , Prev 置空。
- 2 反复选择当前 Dist 最小且未确定的顶点 u 。
- 3 用 u 松弛其邻边, 更新相邻顶点的 Dist 与 Prev。
- 4 全部顶点处理完成后得到最短距离与路径树。

顶点	s	a	b	c	d
Dist	0	4	5	2	4
Prev	-	s	a	s	b



适用条件/复杂度

适用于边权非负图; 基于最小堆实现时复杂度为 $O((|V| + |E|) \log |V|)$ 。

6 Floyd-Warshall: 所有点对最短路径的动态规划

- 定义 $D_k(i, j)$: 只允许经过前 k 个中间顶点时, 从 i 到 j 的最短距离。
- 状态转移:
$$D_{k+1}(i, j) = \min(D_k(i, j), D_k(i, k+1) + D_k(k+1, j))$$
- 三重循环逐步更新距离矩阵, 最终得到所有点对之间的最短距离。

示例 (顶点顺序: 1, 2, 3)

	$D^{(0)}$ (仅考虑直接边)				$D^{(1)}$ (允许中间点 {1})				$D^{(2)}$ (允许中间点 {1, 2})		
	1	2	3		1	2	3		1	2	3
1	0	∞	2		0	∞	2		0	∞	2
2	5	0	8	→	5	0	7	→	5	0	7
3	∞	3	0		∞	3	0		8	3	0



补充说明

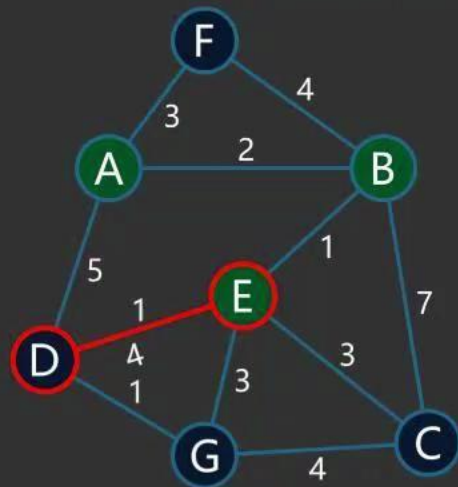
若最终存在 $D[i][i] < 0$, 则图中存在负回路。



时间复杂度 $O(|V|^3)$, 空间复杂度 $O(|V|^2)$ 。

Dijkstra算法

算法动画演示



图论系列

直观理解最短路径搜索全过程



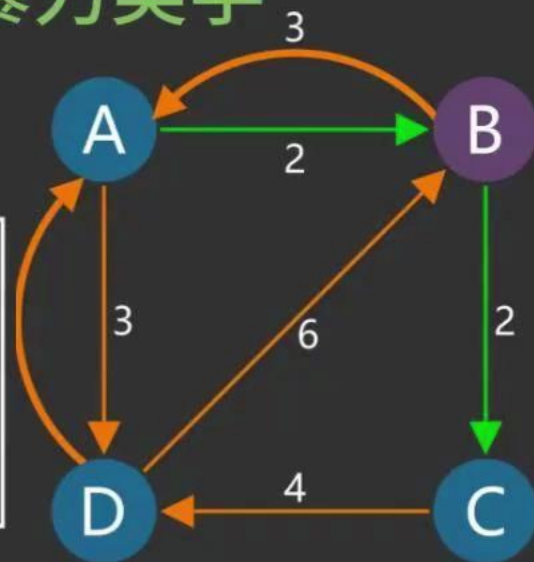
面向初学者|简单又好懂

Floyd-Warshall算法

全能暴力美学

前驱矩阵 T_1

	A	B	C	D
A	—	A	—	A
B	B	—	B	A
C	—	—	—	C
D	D	A	—	—



图论系列

所有节点对之间的最短路径



面向初学者|简单又好懂

“图”的应用

最短路径

Graph

1. Dijkstra 算法的正确性

前提：所有边权非负。

断言：当算法从优先队列中取出顶点 u 时， $\text{dist}[u]$ 已经是源点到 u 的最短距离。

证明（归纳法 + 贪心选择性质）：

- 归纳基础：源点 s 最先被取出， $\text{dist}[s]=0$ ，显然正确。
- 归纳步骤：假设之前取出的所有顶点都已获得最短距离。考虑当前取出的顶点 u ，设其真实最短路径长度为 $d^*(u)$ ，算法给出的距离为 $\text{dist}[u]$ 。
若 $\text{dist}[u] > d^*(u)$ ，则真实最短路径上必然存在第一个不在已处理集合 S 中的顶点 x （紧接在某个 $y \in S$ 之后）。因为边权非负，有：

$$d^*(x) = d^*(y) + w(y, x) \geq d^*(y) \geq \text{dist}[y] \quad (\text{由归纳假设})$$

又因为算法在松弛时已经检查了边 (y, x) ，所以 $\text{dist}[x] \leq \text{dist}[y] + w(y, x) = d^*(x)$ 。于是 $\text{dist}[x] = d^*(x)$ 且 $\text{dist}[x] \leq d^*(u)$ 。由于 x 在真实最短路径上且在 u 之前，故 $d^*(x) < d^*(u)$ （严格小于，除非全零边，但全零边可同样处理）。但算法选择了 u 作为当前最小 dist ，这与 $\text{dist}[x] < \text{dist}[u]$ 矛盾。
因此假设不成立， $\text{dist}[u] = d^*(u)$ 。

结论：贪心选择有效，算法结束时所有顶点的 dist 都是最短距离。

2. Floyd-Warshall 算法的正确性

定义：设 $d^{(k)}[i][j]$ 表示从 i 到 j 仅允许使用前 k 个顶点（编号 $1 \dots k$ ）作为中间点的最短路径长度。初始 $d^{(0)}[i][j] = w(i, j)$ （若无边则为无穷，且 $d^{(0)}[i][i] = 0$ ）。

递推关系：

$$d^{(k)}[i][j] = \min \left(d^{(k-1)}[i][j], d^{(k-1)}[i][k] + d^{(k-1)}[k][j] \right)$$

直观：从 i 到 j 且中间点集限于 $\{1, \dots, k\}$ 的路径，要么不经过 k （前者），要么经过 k （后者分成两段子路径）。

证明（对 k 归纳）：

- 基础 $k = 0$ ：无中间点，直接边，显然正确。
- 归纳假设：对所有 i, j ， $d^{(k-1)}[i][j]$ 是正确的（即允许前 $k - 1$ 个点作为中间点的最短距离）。
- 归纳步骤：考虑任意一条从 i 到 j 且中间点 $\subseteq \{1, \dots, k\}$ 的最短路径 P 。
 - 若 P 不经过 k ，则 P 也是允许 $\{1, \dots, k - 1\}$ 的路径，其长度为 $d^{(k-1)}[i][j]$ 。
 - 若 P 经过 k ，设 k 第一次出现的位置将 P 分为 $i \rightarrow k$ 和 $k \rightarrow j$ 两段。这两段子路径的中间点都不包含 k （否则 k 会重复出现，但最短路径无环，且允许非简单路径？实际上由于边权可能负，但无负环时可假定路径无环；即使有零环，也可取无环版本）。因此两段子路径都是允许 $\{1, \dots, k - 1\}$ 的，长度分别为 $d^{(k-1)}[i][k]$ 和 $d^{(k-1)}[k][j]$ 。总长度为两者之和。
综上， P 的长度等于 $\min(d^{(k-1)}[i][j], d^{(k-1)}[i][k] + d^{(k-1)}[k][j])$ ，与递推式一致。因此 $d^{(k)}[i][j]$ 正确。

最终： $k = V$ 时， $d^{(V)}[i][j]$ 即为允许所有顶点作中间点的最短路径长度，即全局最短距离。

负环检测：若某次迭代后 $\text{dist}[i][i] < 0$ ，则存在负环。

“图”的应用

Graph

最短路径

1. 松弛 (Relaxation) —— 核心操作

所有最短路径算法都在反复执行：

text

复制 下载

```
if dist[u] + w(u,v) < dist[v]:  
    dist[v] = dist[u] + w(u,v)
```

这源于三角不等式：最短路径必须满足 $\text{dist}[v] \leq \text{dist}[u] + w(u,v)$ 。

2. 最优子结构

从 s 到 t 的最短路径，其任何子路径也是对应端点间的最短路径。这一性质支撑了所有算法的正确性。

“图”的应用

Graph

最短路径

3. 动态规划思想

- **Dijkstra**: 贪心 + DP。按距离递增顺序确定每个顶点的最短路径，一旦确定就永不改变（需要非负权保证）。
- **Floyd-Warshall**: 纯粹的 DP。通过逐步允许更多顶点作为中间点来递推。

因此，两者本质上都在求解 **Bellman 最优性方程**：

$$d(v) = \min_{u \in V} \{d(u) + w(u, v)\} \quad (\text{边界 } d(s) = 0)$$

只是求解策略不同。

“图”的应用

Graph

最短路径

维度	Dijkstra	Floyd-Warshall
问题范围	单源最短路径 (SSSP)	全源最短路径 (APSP)
边权限制	非负 (或所有边非负)	允许负权, 但不能有负环 (若有则检测)
策略	贪心: 每次取未处理中距离最小的顶点, 松弛其出边	DP: 依次将每个顶点作为中间点, 尝试改进所有点对距离
数据结构	优先队列 (堆) 或普通数组	二维距离矩阵
时间复杂度	堆: $O((V + E) \log V)$; 数组: $O(V^2)$	$O(V^3)$
空间复杂度	$O(V + E)$	$O(V^2)$
是否能检测负环	不能	能 (检查对角线是否为负)

拓扑排序

“图”的应用

Graph

1 问题引入



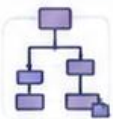
课程先修



项目调度



软件构建



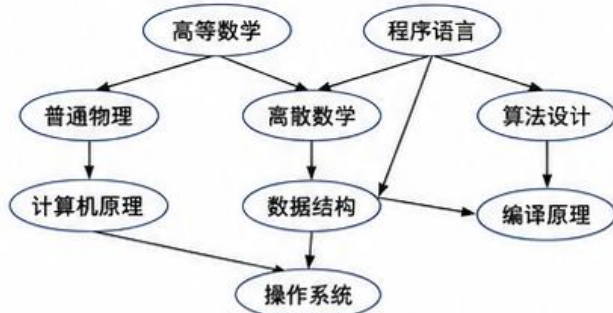
工作流编排

- 许多任务天然存在先后依赖：有些活动必须在另一些活动结束后才能开始。
- 如果只关心先后约束，可建模为 AOV 网；如果还关心持续时间与总工期，可建模为 AOE 网。
- 核心问题：① 是否存在合法执行顺序？② 哪些活动决定最短完工时间？

2 定义与建模

AOV 网与拓扑排序

- AOV 网 (Activity on Vertex Network)：顶点表示活动，边表示活动间的先后关系。
- 拓扑排序：对有向图顶点给出一个线性序列；若存在边 $u \rightarrow v$ ，则 u 必须出现在 v 之前。
- 只有 DAG (有向无环图) 才存在拓扑排序。

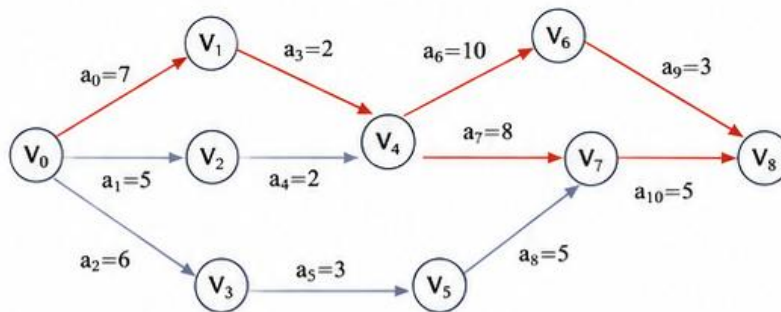


一个合法拓扑排序示例：

高等数学，程序语言，普通物理，离散数学，算法设计，计算机原理，数据结构，编译原理，操作系统

AOE 网与关键路径

- AOE 网 (Activity on Edge Network)：顶点表示事件，边表示活动，边权表示活动持续时间。
- 关键路径：从源点到汇点的最长路径，它决定整个工程的最短完工时间。
- 关键活动一旦延迟，就会导致总工期延长。



→ 关键活动（位于某条关键路径上）

→ 非关键活动

拓扑排序

“图”的应用

Graph

3 关键性质

拓扑排序的性质

1. 入度为 0 的顶点可以立即输出。
2. 每删除一个顶点，都要同步删除其所有出边。
3. 拓扑序通常不唯一。
4. 若处理过程中队列为空但仍有顶点未输出，则图中存在环。
5. 时间复杂度： $O(|V| + |E|)$ 。

关键路径的性质

1. 最早发生时间 $earliest[i]$ 由前驱路径的最大值决定。
2. 最晚发生时间 $latest[i]$ 由后继约束的最小值决定。
3. 活动 (i, j) 的最早开始时间 $ES(i, j) = earliest[i]$ 。
4. 活动 (i, j) 的最晚开始时间 $LS(i, j) = latest[j] - w_{ij}$ 。
5. 若 $ES = LS$ ，则该活动为关键活动。关键路径可能不唯一。
6. 整体时间复杂度同样为 $O(|V| + |E|)$ 。

正向扫描（求 $earliest$ ）

$$earliest[j] \leftarrow \max(earliest[j], earliest[i] + w_{ij})$$

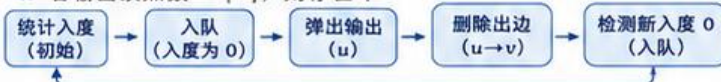
反向扫描（求 $latest$ ）

$$latest[i] \leftarrow \min(latest[i], latest[j] - w_{ij})$$

4 算法

算法一：Kahn 拓扑排序

1. 统计每个顶点入度 $indeg[v]$
2. 将所有入度为 0 的顶点入队
3. while 队列非空：
 - 取出顶点 u 并输出
 - 对每条边 $u \rightarrow v$: $indeg[v]--$
 - 若 $indeg[v] = 0$ ，则将 v 入队
4. 若输出顶点数 $< |V|$ ，则存在环



算法二：关键路径求解

1. 先对 AOE 网做拓扑排序
2. 按拓扑序正向扫描，求 $earliest[]$
3. 将 $latest[]$ 初始化为工程最早完成时间 T
4. 按逆拓扑序反向扫描，求 $latest[]$ 。
5. 对每条活动边计算 ES / LS
6. 取 $ES = LS$ 的活动，连成关键路径

$$\text{其中 } T = \max_i earliest[i]$$



5 示例结果与直观结论

拓扑排序示例



注意：合法拓扑序不唯一。

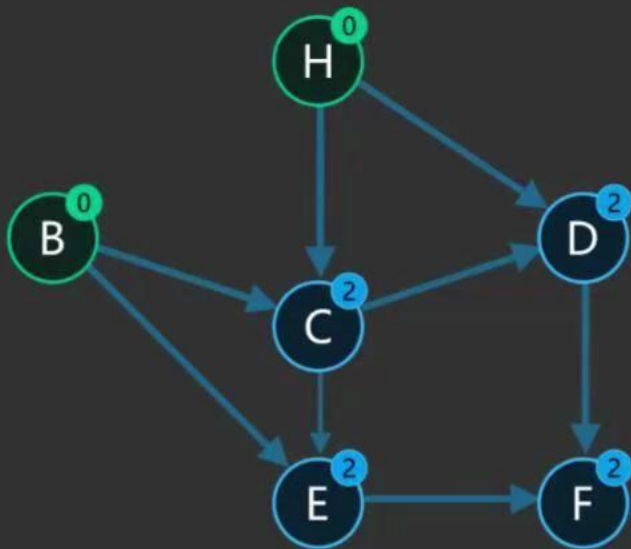
关键路径示例

- 最早完成时间：22
- 事件最早时间： $V_0=0, V_1=7, V_2=5, V_3=6, V_4=9, V_5=9, V_6=19, V_7=17, V_8=22$
- 事件最晚时间： $V_0=0, V_1=7, V_2=7, V_3=9, V_4=9, V_5=12, V_6=19, V_7=17, V_8=22$
- 关键路径 1： $V_0 \rightarrow V_1 \rightarrow V_4 \rightarrow V_6 \rightarrow V_8$
- 关键路径 2： $V_0 \rightarrow V_1 \rightarrow V_4 \rightarrow V_7 \rightarrow V_8$
- 两条关键路径长度均为 22



拓扑排序

Kahn 算法队列版



图论系列

搞懂依赖顺序只要3分钟!



面向初学者|简单又好懂

“图”的应用

Graph

任务	算法	图类型	边权限制	备注
MST	Prim / Kruskal	无向连通	任意实数	负权不影响
SSSP	Dijkstra	有向/无向	非负	
SSSP	Bellman-Ford	有向	可负，无负环	无向图负边 $\Rightarrow$ 负环
SSSP	DAG-SP	DAG	任意	拓扑+DP
APSP	Floyd-Warshall	有向/无向	可负，无负环	
APSP	Johnson	有向	可负，无负环	稀疏图更优
拓扑排序	Kahn / DFS	DAG	无关	

PROGRAM

习题

Practice

DSA小班课 / L4 Practice

🏠

📖

🎓

📅

📶

🕒

💬

📖

L4 Practice

题单简介 题目列表

题数 5 收藏人数 0

题号	题目名称	显示算法 标签	难度	通过率
P1339	[USACO09OCT] Heat Wave G	USACO 福建省历届夏令营 2009	普及/提高-	
P3367	【模板】并查集	模板题	普及/提高-	
P1113	[USACO02FEB] 杂务	USACO 2002	普及/提高-	
P1144	最短路径计数		普及+/提高	
P1330	封锁阳光大学	福建省历届夏令营	普及/提高-	

多选

关于洛谷 · 帮助中心 · 用户协议 · 联系我们 · 小黑屋 · 陶片放逐 · 社区规则

L4 Practice

PROGRAM

Q&A

Q&A